HashiCorp Vault Guide

Architecture, Operations, and Python (`hvac`) Microservice Integration

The Guardian Security & Infrastructure Team

Version 1.0 | November 2026

Abstract

This guide establishes the operational and architectural standards for HashiCorp Vault deployment within the 3-layer urban monitoring platform. It details Raft integrated storage topology, AES-256-GCM encryption at rest, Shamir secret unsealing, AppRole dynamic authentication, KV-v2 versioning, Transit cryptographic acceleration, and automated Python token renewal loops via `hvac`.

1 Architecture and Operations Overview

1.1 Cluster Topology

Vault executes as a high-availability cluster utilizing **integrated storage (Raft consensus)**, eliminating external database dependencies such as Consul or ZooKeeper.

Concept	Operational Mechanism
Raft Consensus	Nodes elect a single active leader. Writes route exclusively through the leader and replicate to followers, committing once a quorum ($n/2+1$ nodes) acknowledges storage.
Fault Tolerance	A 3-node cluster tolerates 1 node outage; a 5-node cluster tolerates 2 outages. Clusters require an odd node count.
Active / Standby	The elected leader serves live requests. Standby nodes forward incoming client requests to the active leader via internal mTLS and execute failover election upon heartbeat timeout.
Network Ports	Port 8200 serves the external client REST API and CLI; Port 8201 serves cluster-internal Raft replication and request forwarding.

1.2 Security Model

- **TLS Transport Enforcement:** The API listener enforces HTTPS. Client microservices must pass a verified CA bundle (`VAULT_CACERT`) to validate server hostnames. Disabling certificate verification (`verify=False`) is strictly forbidden.
- **Envelope Encryption at Rest:** Storage records are encrypted with AES-256-GCM using Vault's internal encryption key, which is itself wrapped by the cluster root key. Stolen disks or snapshot volumes yield only ciphertext.
- **Shamir Secret Unsealing:** Following restart, Vault enters a sealed state. Unsealing requires a threshold quorum of key shares (e.g., 3-of-5 threshold) generated during cluster initialization. Unseal shares must be partitioned across administrative officers and stored in HSMs or encrypted keychains.
- **Engines Employed:**
  - **KV v2 (`secret/`):** Versioned secret storage for database credentials and service parameters.

  - **Transit (`transit/`):** Cryptography-as-a-Service providing RSA-4096 and Ed25519 offloading without exposing private key material to client applications.

- **Audit Logging:** File-based audit devices log all cryptographic operations and access attempts to `/vault/logs/audit.log`.

1.3 AppRole Authentication Workflow

Client microservices authenticate via AppRole:

1. **Policy Definition:** Administrators define path-scoped HCL policies enforcing default-deny semantics.
2. **Credential Provisioning:** The role yields a static `role_id` (application identifier) and a dynamically injected, short-lived `secret_id` (mounted via Docker secret).
3. **Token Issuance:** The application exchanges credentials at `auth/approle/login`, receiving a 15-minute scoped client token.

2 Python (`hvac`) Integration Guide

2.1 Client Initialization and AppRole Login

```
import os, hvac

def make_client() -> hvac.Client:
    return hvac.Client(
        url=os.environ["VAULT_ADDR"],
        verify=os.environ.get("VAULT_CACERT", True),
        timeout=10,
    )

def login(client: hvac.Client) -> int:
    response = client.auth.approle.login(
        role_id=os.environ["VAULT_ROLE_ID"],
        secret_id=os.environ["VAULT_SECRET_ID"],
    )
    if not client.is_authenticated():
        raise RuntimeError("AppRole login failed: token invalid")
    return response["auth"]["lease_duration"]
```

2.2 Background Token Renewal Daemon

To prevent service outages upon token expiration, microservices utilize a background renewal thread:

```
import threading, logging

class VaultSession:
    def __init__(self):
        self.client = make_client()
        self._ttl = login(self.client)
        self._stop = threading.Event()
        self._thread = threading.Thread(target=self._renew_loop, daemon=True)
        self._thread.start()

    def _renew_loop(self):
        while not self._stop.wait(max(self._ttl * 2 / 3, 1)):
            try:
                lease = self.client.auth.token.renew_self()["auth"]["lease_duration"]
                self._ttl = lease
            except Exception:
                self._ttl = login(self.client)
```

```
    def close(self):
        self._stop.set()
        self._thread.join(timeout=5)
        try:
            self.client.auth.token.revoke_self()
        finally:
            self.client.adapter.close()
```

3 Transit Decryption Service

Departmental RSA-4096 private keys remain sealed inside Vault. Services forward encrypted payloads in Transit format (`vault:v1:<base64>`) and receive plaintext bytes:

```
import base64

def decrypt_for_department(client: hvac.Client, department: str, ciphertext: str) ->
    bytes:
    response = client.secrets.transit.decrypt_data(
        name=f"dept-{department}",
        ciphertext=ciphertext,
        mount_point="transit",
    )
    return base64.b64decode(response["data"]["plaintext"])
```

4 Security Verification Checklist

✓ TLS 1.3 enforced on listener; client CA bundle verification enabled.

✓ Initial root token revoked; Shamir shares distributed across authorized officers.

✓ AppRole `secret_id` injected via runtime secrets, absent from Git and container images.

✓ Path-level least-privilege policies active for `guardian-app`, `twin-app`, and `brain-app`.

✓ Audit device writing structured audit records to persistent volume.